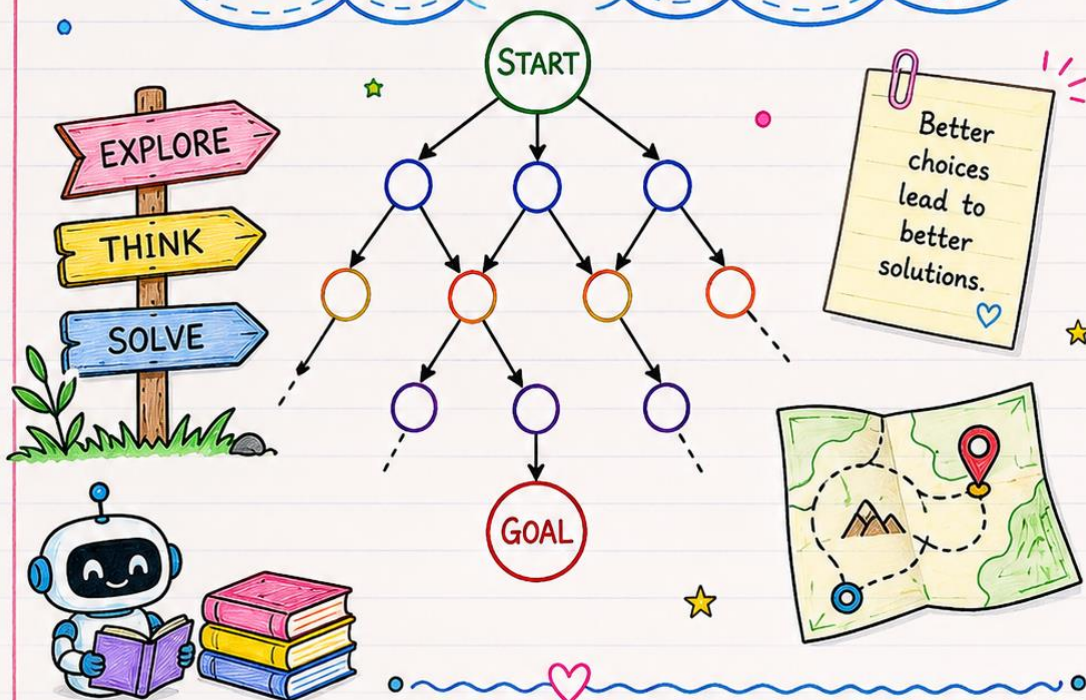




UNIT 2



State Space Search and Heuristic Technique:

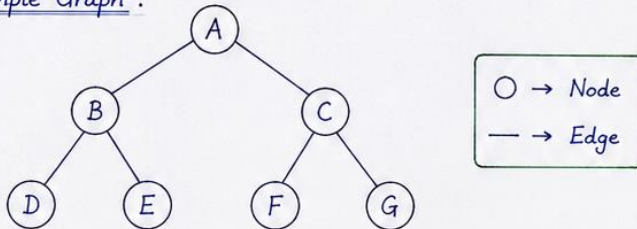


Q.3 (7 Marks) Illustrate Breadth First Search (BFS) with an example.

Page-3

Ans: Breadth First Search (BFS) is a graph traversal algorithm. It explores the graph level by level (breadth-wise) starting from the source node. It uses a Queue (FIFO) to keep track of nodes to be visited.

Example Graph:



Given:

- Vertices : {A, B, C, D, E, F, G}
- Edges : {(A, B), (A, C), (B, D), (B, E), (C, F), (C, G)}
- Start node (Source) : A
- Goal : Visit all nodes

BFS Algorithm (Steps):

1. Start from the source node.
2. Mark it as visited and insert it into the queue.
3. While the queue is not empty :
 - Remove the front node from the queue.
 - Visit all its unvisited neighbors, mark them visited and insert them into the queue.

Key Idea:

First visit all neighbors at the current level, then move to the next level.

Step-by-Step BFS Traversal from A:

Step	Queue (Front → Rear)	Visited	Action / Explanation
1	[A]	{A}	Start at A. Enqueue A.
2	[B, C]	{A, B, C}	Dequeue A. Visit A's neighbors B, C. Enqueue B, C.
3	[D, E, F, G]	{A, B, C, D, E, F, G}	Dequeue B. Visit B's neighbors D, E. Enqueue D, E.
4	[F, G]	{A, B, C, D, E, F, G}	Dequeue C. Visit C's neighbors F, G. Enqueue F, G.
5	[G]	{A, B, C, D, E, F, G}	Dequeue D. No new neighbor.
6	[]	{A, B, C, D, E, F, G}	Dequeue E, F, G. No new neighbors. (Queue becomes empty. Stop.)

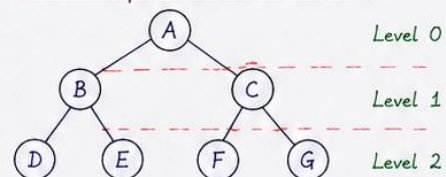
BFS Traversal Order:

A → B → C → D → E → F → G

Level-wise view:

Level 0 : A
Level 1 : B, C
Level 2 : D, E, F, G

Visual Representation (Levels):



Applications of BFS:

- * Finding the shortest path in an unweighted graph.
- * Web crawling (search engines).
- * Social network analysis (finding degrees of connection).
- * Network broadcasting (sending messages to all nodes).

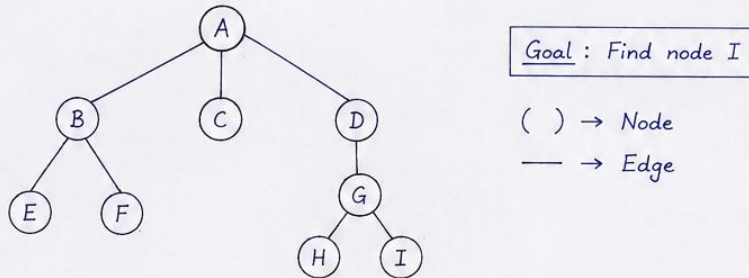
Conclusion:

BFS explores the graph level by level using a Queue (FIFO). It guarantees the shortest path (in terms of number of edges) in an unweighted graph. 😊

Q.2 (7 Marks) Demonstrate Depth First Search (DFS) with an example. Page-2

Ans: Depth First Search (DFS) is an uninformed search algorithm. It explores as deep as possible along one branch before backtracking to explore other branches.

Example: Consider the following graph.



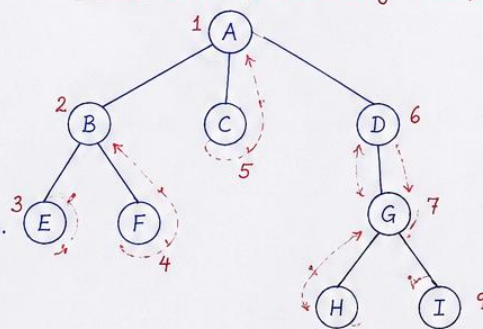
Step-by-Step DFS Traversal (Start from A)

Step	Stack (Top → Right)	Action	Visited Order
1	A	Push A (start)	A
2	D, C, B	Pop A, push its children (B,C,D)	A
3	D, C, F, E	Pop B, push its children (E,F)	A, B
4	D, C, F	Pop E (no child)	A, B, E
5	D, C	Pop F (no child)	A, B, E, F
6	D	Pop C (no child)	A, B, E, F, C
7	G	Pop D, push its child (G)	A, B, E, F, C, D
8	I, H	Pop G, push its children (H,I)	A, B, E, F, C, D, G
9	I	Pop H (no child)	A, B, E, F, C, D, G, H
10	—	Pop I → Goal Found	A, B, E, F, C, D, G, H, I

DFS Traversal Order:

A → B → E → F → C → D → G → H → I

Visual DFS (Backtracking shown)



Key Points:

- DFS uses a Stack (LIFO).
- It goes as deep as possible before backtracking.
- Less memory than BFS.
- Not guaranteed to find shortest path.

Applications of DFS:

- * Solving puzzles (e.g., 8-puzzle, maze).
- * Topological sorting.
- * Detecting cycles in a graph.
- * Finding connected components.

Conclusion:

DFS explores a path completely before backtracking. It is useful when solution may be far from the start or when we want to explore all possibilities.



Q.1 (7 Marks) Explain State Space Search with a suitable example.

Page - 1

Ans: State Space Search is a technique used in AI to solve problems by systematically exploring all possible states to find a path from the initial state to a goal state.

1) State Space :

- A state space is the set of all possible states that can be reached from the initial state by applying a sequence of actions.
- It is usually represented as a graph where nodes are states and edges are actions.

2) Components of State Space Search :

- Initial State : The state at which the problem begins.
- Actions : Set of operations that can be applied.
- Transition Model : Result of applying an action to a state.
- Goal Test : Determines whether a state is a goal state.
- Path Cost : Cost of a path is the sum of costs of actions.

Example : (8-Puzzle Problem)

- We have a 3x3 board with 8 tiles and one blank.
- The task is to reach the goal configuration from the start configuration by sliding tiles into the blank space.

Initial State

1	2	3
4	5	6
7	-	8

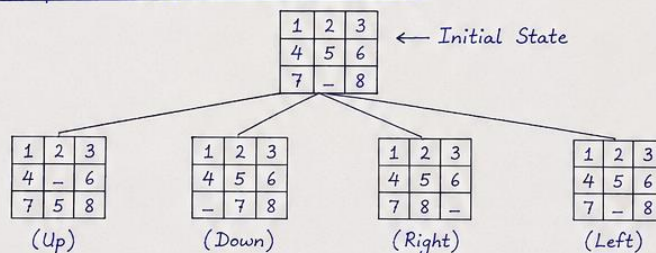
Goal State

1	2	3
4	5	6
7	8	-

Possible Actions

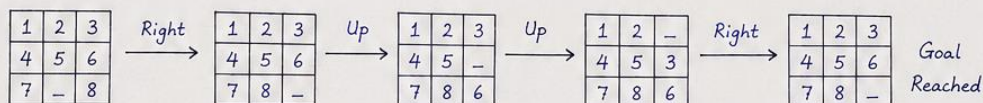
1. Up	2. Down
3. Left	4. Right

State Space Tree (From Initial State):



Each node represents a state. From each state, different actions generate new states. The search algorithm explores this tree to find the goal.

Solution Path Example :



Path Cost = 4 (Assuming each move cost = 1)

Conclusion :

State Space Search systematically explores all possible states of a problem using actions. It is the foundation of many search algorithms like BFS, DFS, Uniform Cost Search, etc.

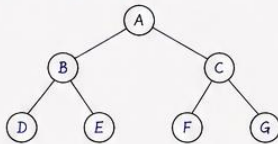
Q.4 (7 Marks) Differentiate Depth First Search (DFS) and Breadth First Search

Page-4

Ans: DFS and BFS are two fundamental uninformed search algorithms used in AI to explore a graph. They differ in the way nodes are visited and in their performance.

Feature	Depth First Search (DFS)	Breadth First Search (BFS)
1. Traversal Style	Explores as deep as possible along one branch before backtracking.	Explores all neighbors level by level (breadth-wise).
2. Data Structure	Stack (LIFO)	Queue (FIFO)
3. Strategy	Go deep first, then backtrack.	Visit all nodes at current level before going to next level.
4. Completeness	Not complete in infinite depth or cycles (may get stuck).	Complete (finds solution if it exists in finite time).
5. Optimality	Not optimal.	Optimal for unweighted graphs (shortest path in terms of edges).
6. Time Complexity	$O(b^m)$	$O(b^d)$
7. Space Complexity	$O(bm)$	$O(b^d)$
8. Best Suited For	Problems with large search space but small solution depth.	Problems where shortest path is required.
9. Memory Usage	Less memory (stores one path from root to current node).	More memory (stores all nodes at current level).
10. Backtracking	Uses backtracking.	No backtracking.
11. Example	Maze solving, Topological sorting, Cycle detection.	Shortest path in unweighted graph, Web crawling, Social network search.

Example Graph (Same for DFS and BFS):



Given:

Start node : A

Goal : (Assume any node)

(We will visit all nodes)

Legend:

○ → Node

— → Edge

Depth First Search (DFS) from A

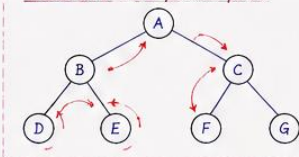
(Using Stack)

Traversal Order : A → B → D → E → C → F → G

Step-by-Step (Stack):

Step	Stack (Top → Right)	Visited	Action
1	A	A	Push A
2	C, B	A	Push B, C
3	C, E, D	A, B	Push D, E
4	C, E	A, B, D	D has no child → Pop D
5	C	A, B, D, E	E has no child → Pop E
6	F, G	A, B, D, E, C	Push F, G
7	F	A, B, D, E, C, G	G has no child → Pop G
8	— (Done)	A, B, D, E, C, G, F	F has no child → Pop F

Visual DFS (One possible path)



Key Differences in One Line:

DFS dives deep into one branch before exploring others.

BFS explores all neighbors at the current level first.

Breadth First Search (BFS) from A

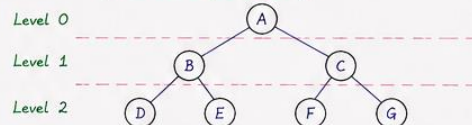
(Using Queue)

Traversal Order : A → B → C → D → E → F → G

Step-by-Step (Queue):

Step	Queue (Front → Rear)	Visited	Action
1	[A]	A	Enqueue A
2	[B, C]	A, B, C	Dequeue A Enqueue B, C
3	[D, E, F, G]	A, B, C, D, E, F, G	Dequeue B Enqueue D, E Dequeue C Enqueue F, G
4	[F, G]	A, B, C, D, E, F, G	Dequeue D, E
5	[G]	A, B, C, D, E, F, G	No new node
6	[]	A, B, C, D, E, F, G	No new node
7	[]	A, B, C, D, E, F, G	No new node
8	[] (Done)	A, B, C, D, E, F, G	No new node

Visual BFS (Level-wise)



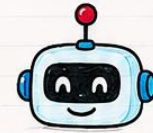
Conclusion:

Both DFS and BFS are important. DFS is useful when memory is limited and solution depth is small, while BFS is optimal for finding the shortest path in an unweighted graph. 😊



Heuristic Function

(Important in Artificial Intelligence)



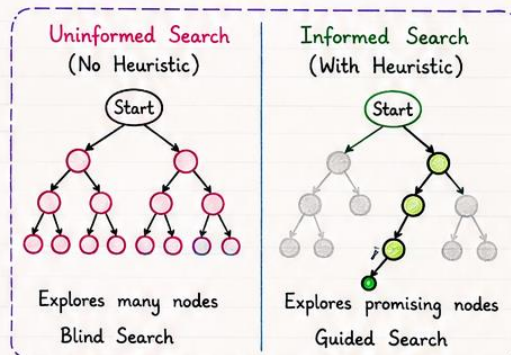
1. What is a Heuristic Function?

- A **heuristic function** ($h(n)$) is a function used in search algorithms (especially in AI) to **estimate** the **cost** of reaching the goal from a given **state** n .
- It provides **guidance** to the search process by making it more **efficient**.
- Heuristics are like "**rules of thumb**" – they don't guarantee the best solution but often lead to **good/optimal solutions** faster.

“Heuristic =
Intelligent Guess
to Reach Goal”

2. Role of Heuristic Function

- In **uninformed search** (like BFS, DFS), the algorithm has no knowledge of the problem domain; it **blindly explores**.
- In **informed search** (heuristic search), a heuristic function helps decide which node to explore first, **reducing time and memory usage**.



3. Example: 8-Puzzle Problem

- Goal:** Arrange tiles in correct order.
- Two common heuristic functions:
 - $h_1(n)$: Number of **misplaced tiles** (tiles not in the correct place).
 - $h_2(n)$: **Manhattan distance** (sum of distances of each tile from its goal position).

Both help guide the search, but **Manhattan distance** is more accurate.

Initial State	Goal State	Heuristic Values																		
<table border="1"> <tr><td>2</td><td>8</td><td>3</td></tr> <tr><td>1</td><td>6</td><td>4</td></tr> <tr><td>7</td><td>—</td><td>5</td></tr> </table>	2	8	3	1	6	4	7	—	5	<table border="1"> <tr><td>1</td><td>2</td><td>3</td></tr> <tr><td>4</td><td>5</td><td>6</td></tr> <tr><td>7</td><td>8</td><td>—</td></tr> </table>	1	2	3	4	5	6	7	8	—	$h_1(n)$ Misplaced Tiles Tiles misplaced: 2, 8, 1, 6, 4, 7, 5 $\Rightarrow h_1(n) = 7$
2	8	3																		
1	6	4																		
7	—	5																		
1	2	3																		
4	5	6																		
7	8	—																		
$h_2(n)$ Manhattan Distance $2(1) + 8(1) + 1(1) + 6(1) + 4(1) + 7(1) + 5(1) = 7$ $\Rightarrow h_2(n) = 7$																				

(X) = distance of tile from goal position

4. Properties of a Good Heuristic

- Admissibility** $\rightarrow$ Never overestimates the true cost. (Guarantees optimal solution in A* search).
- Consistency (Monotonicity)** $\rightarrow$ Estimate should be non-decreasing along a path.

For every node n and its successor n' ,
 $h(n) \leq c(n, n') + h(n')$
 where $c(n, n')$ = cost from n to n' .



5. Applications of Heuristic Function

★ **A* Algorithm** ($f(n) = g(n) + h(n)$)

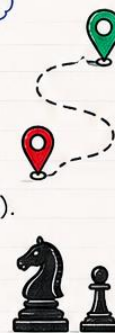
- $g(n)$ = cost so far
- $h(n)$ = estimated cost to goal

★ **Greedy Best-First Search** ($f(n) = h(n)$)

★ **Pathfinding** (Google Maps, GPS systems).

★ **Game playing** (Chess, Tic-tac-toe evaluation functions).

★ **AI planning & optimization problems.**



In short: Heuristic functions add **intelligence** to search algorithms by providing **direction** towards the goal, making them **faster** and **smarter**!

